Tool API Reference: Kubernetes Multi-Agent Troubleshooter

Date: 2026-11-04

Overview

The three agents call 11 read-only tools. This reference defines each tool's inputs, the API behind it, and the result it returns.

Target platform: Kubernetes 1.34.4, the `kubernetes` Python client, and the Prometheus HTTP API. No tool changes cluster state.

Tool	Agent	Backing API	Required input
`list_nodes`	Node health	`GET /api/v1/nodes`	none
`describe_node`	Node health	`GET /api/v1/nodes/{name}` plus events	`name`
`node_events`	Node health	`GET /api/v1/events` (field selector)	`node`
`pods_per_node`	Node health	`GET /api/v1/pods` (field selector)	`node`
`get_requests_limits`	Right-sizing	`GET /api/v1/pods`	none
`query_cpu_usage`	Right-sizing	Prometheus `GET /api/v1/query`	none
`query_memory_usage`	Right-sizing	Prometheus `GET /api/v1/query`	none
`list_pods`	Pod security	`GET /api/v1/pods`	none
`get_pod_spec`	Pod security	`GET /api/v1/namespaces/{ns}/pods/{name}`	`namespace`, `name`
`check_security_context`	Pod security	none (rules applied to pod specs)	`namespace`
`find_latest_tags`	Pod security	none (rules applied to pod specs)	none

Conventions used by every tool:

- Inputs are JSON objects. Unknown fields are rejected with `INVALID_INPUT`.
- CPU is returned in millicores and memory in MiB, so agents never parse Kubernetes quantity strings.
- Timestamps are RFC 3339 in UTC.
- Every result uses one envelope, defined under Result envelope and errors.

Node health tools

These four tools let the node health agent find an unhealthy node, read its conditions and events, and check pod density.

list_nodes

Lists nodes with a compact health summary. API: `GET /api/v1/nodes` (`CoreV1Api.list_node`).

Input	Type	Default	Meaning
`label_selector`	string	none	Standard label selector, for example `node-role.k
`only_unhealthy`	boolean	false	Return only nodes that are NotReady, unschedu

Result `data.nodes[]` fields: `name`, `ready`, `roles`, `unschedulable`, `failing_conditions`, `taints`, `kubelet_version`, `os_image`, `cpu_allocatable_m`, `mem_allocatable_mib`, `pods_allocatable`. Also `data.count`.


```
{"ok": true, "tool": "list_nodes", "data": {"count": 1, "nodes": [{
  "name": "node-3", "ready": false, "roles": ["worker"], "unschedulable": false,
  "failing_conditions": ["DiskPressure"], "taints": ["node.kubernetes.io/disk-pressure:NoSchedule"],
  "kubelet_version": "v1.34.4", "cpu_allocatable_m": 3920, "mem_allocatable_mib": 15200, "pods_allocatable": 110}]}
 "truncated": false, "error": null}
```


describe_node

Returns one node's conditions, capacity, taints, and recent events, the same material as `kubectl describe node`. API: `GET /api/v1/nodes/{name}` plus `GET /api/v1/events` filtered to that node.

Input	Type	Required	Meaning
`name`	string	yes	Node name
`include_usage`	boolean	no, default true	Add live usage from `metrics.k8s.io` when metric

Result `data` fields: `name`, `conditions[]` (`type`, `status`, `reason`, `message`, `last_transition`), `taints`, `capacity`, `allocatable`, `node_info` (`kubelet_version`, `kernel_version`, `container_runtime`, `os_image`), `unschedulable`, `usage` (`cpu_m`, `mem_mib`, or null), `recent_events` (last 10, same shape as `node_events`).


```
{"ok": true, "tool": "describe_node", "data": {"name": "node-3",
  "conditions": [{"type": "DiskPressure", "status": "True", "reason": "KubeletHasDiskPressure",
    "message": "kubelet has disk pressure", "last_transition": "2026-11-04T08:12:41Z"}],
  "usage": {"cpu_m": 1210, "mem_mib": 9870}, "recent_events": []}, "truncated": false, "error": null}
```


node_events

Lists recent events for one node, newest first. API: `GET /api/v1/events?fieldSelector=involvedObject.kind=Node,involvedObject.name={node}` (`list_event_for_all_namespaces`).

Input	Type	Default	Meaning
`node`	string	required	Node name
`warnings_only`	boolean	false	Keep only events of type `Warning`
`since_minutes`	integer	120	Ignore events older than this
`limit`	integer	30 (max 100)	Maximum events returned

Result `data.events[]` fields: `type`, `reason`, `message`, `count`, `first_seen`, `last_seen`, `source` (reporting component). Typical reasons: `NodeNotReady`, `EvictionThresholdMet`, `ImageGCFailed`, `FreeDiskSpaceFailed`.

pods_per_node

Counts pods on a node and compares the count with the node's pod limit. API: `GET /api/v1/pods?fieldSelector=spec.nodeName={node}` and `GET /api/v1/nodes/{node}` for `status.allocatable.pods`.

Input	Type	Default	Meaning
`node`	string	required	Node name
`include_pods`	boolean	false	Also return the pod list (namespace, name, phas

Result `data` fields: `node`, `pod_count`, `max_pods`, `utilization_pct`, `by_phase` (counts per phase), `top_namespaces` (up to 10), and `pods[]` when requested.

```
{"ok": true, "tool": "pods_per_node", "data": {"node": "node-3", "pod_count": 104,
  "max_pods": 110, "utilization_pct": 94.5, "by_phase": {"Running": 101, "Pending": 2, "Failed": 1},
  "top_namespaces": [{"namespace": "prod", "count": 61}]}, "truncated": false, "error": null}
```

Common errors for all four tools: `NOT_FOUND` (unknown node), `FORBIDDEN` (missing RBAC), `UNAVAILABLE` (API server timeout).

Right-sizing tools

These three tools return what each container asks for and what it actually uses. The agent compares the two and proposes new values; no tool computes a recommendation itself.

get_requests_limits

Returns configured CPU and memory requests and limits per container. API: `GET /api/v1/pods` or `GET /api/v1/namespaces/{ns}/pods`, reading `spec.containers[].resources` and `spec.initContainers[].resources`. Pod-level `spec.resources` (beta in 1.34) is returned when set.

Input	Type	Default	Meaning
`namespace`	string	all namespaces	Limit to one namespace
`label_selector`	string	none	Standard label selector
`only_missing`	boolean	false	Return only containers with no request or no limi
`limit`	integer	200 (max 500)	Maximum containers returned

Result `data.containers[]` fields: `namespace`, `pod`, `container`, `owner` (`kind`, `name`), `qos_class`, `cpu_request_m`, `cpu_limit_m`, `mem_request_mib`, `mem_limit_mib`, `missing` (list such as `cpu_limit`). An unset value is null.

```
{"ok": true, "tool": "get_requests_limits", "data": {"containers": [{
  "namespace": "prod", "pod": "api-7d9f-x2", "container": "api",
  "owner": {"kind": "Deployment", "name": "api"}, "qos_class": "Burstable",
  "cpu_request_m": 1000, "cpu_limit_m": 2000, "mem_request_mib": 2048, "mem_limit_mib": 2048,
  "missing": []}]}, "truncated": false, "error": null}
```

query_cpu_usage

Returns observed CPU usage per container over a lookback window. API: Prometheus `GET /api/v1/query` (instant query with a subquery), using cAdvisor metrics.

Input	Type	Default	Meaning
`namespace`	string	all	Namespace filter
`pod_regex`	string	`.+`	Pod name regular expression
`container`	string	any	Container name filter
`stat`	string	`p95`	One of `avg`, `p95`, `max`
`lookback`	string	`7d`	How far back to evaluate; maximum `30d`
`rate_window`	string	`5m`	Window for the CPU rate
`limit`	integer	200	Maximum series returned

PromQL the tool builds for `p95` (filters inserted from the inputs):

```
quantile_over_time(0.95,
  (sum by (namespace, pod, container) (
     rate(container_cpu_usage_seconds_total{container!="", container!="POD"}[5m])))[7d:5m])
```

Result `data.series[]` fields: `namespace`, `pod`, `container`, `value_m` (millicores), `stat`, `lookback`. `data.promql` echoes the query so the agent can quote it. An optional field `throttled_ratio` comes from `container_cpu_cfs_throttled_periods_total` divided by `container_cpu_cfs_periods_total`.

```
{"ok": true, "tool": "query_cpu_usage", "data": {"stat": "p95", "lookback": "7d",
  "series": [{"namespace": "prod", "pod": "api-7d9f-x2", "container": "api",
    "value_m": 180, "throttled_ratio": 0.01}]}, "truncated": false, "error": null}
```

In this example the container requests 1000m and uses 180m at p95, so the agent can flag it as over-provisioned.

query_memory_usage

Returns observed memory use per container. API: Prometheus `GET /api/v1/query`, using `container_memory_working_set_bytes`, the value the kernel's OOM killer acts on.

Input	Type	Default	Meaning
`namespace`	string	all	Namespace filter
`pod_regex`	string	`.+`	Pod name regular expression
`container`	string	any	Container name filter

Input	Type	Default	Meaning
`stat`	string	`max`	One of `avg`, `p95`, `max`
`lookback`	string	`7d`	How far back to evaluate; maximum `30d`
`limit`	integer	200	Maximum series returned

PromQL the tool builds for `max`:

```
max_over_time(
  (max by (namespace, pod, container) (
     container_memory_working_set_bytes{container!="", container!="POD"}))[7d:5m])
```

Result `data.series[]` fields: `namespace`, `pod`, `container`, `value_mib`, `stat`, `lookback`, and `oom_killed_recently` (true when `kube_pod_container_status_last_terminated_reason{reason="OOMKilled"}` is 1).

```
{"ok": true, "tool": "query_memory_usage", "data": {"stat": "max", "lookback": "7d",
  "series": [{"namespace": "prod", "pod": "api-7d9f-x2", "container": "api",
    "value_mib": 1890, "oom_killed_recently": false}]}, "truncated": false, "error": null}
```

Common errors: `INVALID_INPUT` (bad regex or duration), `PROMETHEUS_ERROR` (query failed or timed out), `NO_DATA` (the metric returned no series; check that cAdvisor and kube-state-metrics are scraped).

Pod security tools

Two tools fetch pod data from the API. The other two are rule engines that run over that data and map findings to the Pod Security Standards (PSS). PodSecurityPolicy no longer exists in 1.34, so Pod Security Admission labels are the enforcement baseline.

list_pods

Lists pods with the fields needed to pick targets for a scan. API: `GET /api/v1/pods` or `GET /api/v1/namespaces/{ns}/pods`, with `limit` and `continue` paging.

Input	Type	Default	Meaning
`namespace`	string	all namespaces	Limit to one namespace
`label_selector`	string	none	Standard label selector
`node`	string	none	Only pods on this node (`spec.nodeName` field s
`phase`	string	any	`Running`, `Pending`, `Failed`, `Succeeded`
`limit`	integer	200 (max 500)	Page size
`continue_token`	string	none	Token from the previous page

Result `data.pods[]` fields: `namespace`, `name`, `node`, `phase`, `owner` (`kind`, `name`), `containers` (names), `images`, `started_at`. `data.continue_token` is set when more pages exist.

get_pod_spec

Returns the security-relevant parts of one pod's spec, with `managedFields` and other noise removed. API: `GET /api/v1/namespaces/{namespace}/pods/{name}`, plus `GET /api/v1/namespaces/{namespace}` for Pod Security Admission labels.

Input	Type	Required	Meaning
`namespace`	string	yes	Pod namespace
`name`	string	yes	Pod name
`include_status`	boolean	no, default false	Add container statuses and conditions

Result `data` fields: `namespace`, `name`, `service_account`, `host_network`, `host_pid`, `host_ipc`, `pod_security_context`, `volumes[]` (`name`, `type`, `host_path` when set), `containers[]`, `init_containers[]`, `ephemeral_containers[]`, and `psa_labels` (`enforce`, `audit`, `warn`, each with the level and version). Each container entry holds `name`, `image`, `image_pull_policy`, `security_context`, `resources`, and `ports` (with `host_port` when set).


```
{"ok": true, "tool": "get_pod_spec", "data": {"namespace": "prod", "name": "api-7d9f-x2",
  "host_network": false, "psa_labels": {"enforce": "baseline:latest", "audit": null, "warn": null},
  "containers": [{"name": "api", "image": "registry.local/api:latest",
    "security_context": {"privileged": true, "runAsNonRoot": null}}]}, "truncated": false, "error": null}
```


check_security_context

Evaluates pods against the PSS rules below and returns findings. It has no API of its own: it calls `get_pod_spec` for one pod, or `list_pods` plus `get_pod_spec` for a namespace scan.

Input	Type	Default	Meaning
`namespace`	string	required	Namespace to check
`name`	string	none	Check one pod; omit to scan the whole namespa
`label_selector`	string	none	Narrow a namespace scan
`level`	string	`baseline`	Report violations of `baseline` only, or of `restrict
`max_pods`	integer	100	Cap on pods scanned

Rules applied:

Rule id	Field checked	Fails when	PSS level	Severity
`PRIVILEGED`	`securityContext.privileged`	true	baseline	high
`HOST_NAMESPACES`	`hostNetwork`, `hostPID`, `hostIPC`	any is true	baseline	high
`HOST_PATH`	`volumes[].hostPath`	any hostPath volume exists	baseline	high
`HOST_PORT`	`ports[].hostPort`	set to a non-zero value	baseline	medium
`CAPABILITIES_ADDED`	`capabilities.add`	adds anything beyond `NET_BIND_SERVICE`	baseline	high
`RUN_AS_ROOT`	`runAsNonRoot`, `runAsUser`	`runAsNonRoot` is not true, or `runAsUser` is 0	restricted	medium

Rule id	Field checked	Fails when	PSS level	Severity
`PRIV_ESCALATION`	`allowPrivilegeEscalation`	not explicitly false	restricted	medium
`CAPABILITIES_NOT_DROPPED`	`capabilities.drop`	does not drop `ALL`	restricted	low
`SECCOMP_MISSING`	`seccompProfile.type`	not `RuntimeDefault` or `Localhost`	restricted	low

Every rule is checked on `containers`, `initContainers`, and `ephemeralContainers`; pod-level settings are merged with container-level ones before the check.

Result `data` fields: `summary` (`pods_scanned`, `findings_by_severity`), and `findings[]` with `namespace`, `pod`, `container`, `rule`, `severity`, `level`, `field_path`, `observed`, `message`, `hint`. `psa_enforced_level` shows what the namespace already enforces.


```
{"ok": true, "tool": "check_security_context", "data": {
  "summary": {"pods_scanned": 1, "findings_by_severity": {"high": 1, "medium": 1}},
  "psa_enforced_level": "baseline",
  "findings": [{"namespace": "prod", "pod": "api-7d9f-x2", "container": "api", "rule": "PRIVILEGED",
    "severity": "high", "level": "baseline", "field_path": "spec.containers[0].securityContext.privileged",
    "observed": true, "message": "Container runs privileged", "hint": "Remove privileged or grant only the needed c
 "truncated": false, "error": null}
```


find_latest_tags

Finds container images that are not pinned to a fixed version. It has no API of its own: it reads images from pod specs returned by `list_pods`.

Input	Type	Default	Meaning
`namespace`	string	all namespaces	Limit to one namespace
`label_selector`	string	none	Standard label selector
`require_digest`	boolean	false	Also report tagged images that have no `@sha25

Issue types returned:

Issue	Meaning
`LATEST_TAG`	The tag is `latest`
`NO_TAG`	No tag and no digest, which Kubernetes treats as `latest`
`NO_DIGEST`	Tagged but not digest-pinned (only with `require_digest`)

Result `data` fields: `summary` (`images_scanned`, `counts_by_issue`) and `images[]` with `namespace`, `pod`, `container`, `image`, `issue`, `image_pull_policy`. An image pinned by digest never produces a finding.


```
{"ok": true, "tool": "find_latest_tags", "data": {"summary": {"images_scanned": 212, "counts_by_issue": {"LATEST_TA
  "images": [{"namespace": "prod", "pod": "api-7d9f-x2", "container": "api",
    "image": "registry.local/api:latest", "issue": "LATEST_TAG", "image_pull_policy": "Always"}]},
 "truncated": false, "error": null}
```


Common errors: `NOT_FOUND` (unknown pod or namespace), `FORBIDDEN` (missing RBAC), `INVALID_INPUT` (a `name` given without a `namespace`).

Result envelope and errors

Every tool returns the same JSON envelope, so the agent reads success, truncation and failure the same way for all 11 tools.

```
{"ok": true,
 "tool": "list_nodes",
 "data": { },
 "truncated": false,
 "error": null}
```

On failure, `ok` is false, `data` is null, and `error` carries the cause:

```
{"ok": false, "tool": "describe_node", "data": null, "truncated": false,
 "error": {"code": "NOT_FOUND", "message": "node \"node-9\" not found", "retryable": false}}
```

Error code	Raised when	Retryable
`INVALID_INPUT`	A field is missing, has the wrong type, is unknown, or exceeds a maximum	no
`NOT_FOUND`	The node, pod or namespace does not exist (HTTP 404)	no
`FORBIDDEN`	The ServiceAccount lacks RBAC for the call (HTTP 403)	no
`UNAVAILABLE`	The API server timed out or returned 5xx or 429	yes
`PROMETHEUS_ERROR`	Prometheus rejected the query, timed out, or was unreachable	yes
`NO_DATA`	A PromQL query returned no series	no

When a result is cut to fit a limit, `truncated` is true and `data` names what was dropped (for example `data.omitted_count`). The agent should narrow the filters rather than ask for more.

These limits are suggested defaults; change them in the tool configuration.

Limit	Default
Kubernetes API call timeout	15 seconds
Prometheus query timeout	30 seconds
Retries on `UNAVAILABLE`	2, with exponential backoff
Maximum items in any list result	500
Maximum serialized result size	20 KB; larger results are truncated

Permissions and prerequisites

One read-only ClusterRole covers every Kubernetes call. Prometheus is reached over HTTP and needs no RBAC.

Tool	Kubernetes resources	Verbs
`list_nodes`	`nodes`	list

Tool	Kubernetes resources	Verbs
`describe_node`	`nodes`, `events`, `nodes.metrics.k8s.io` (optional)	get, list
`node_events`	`events`	list
`pods_per_node`	`pods`, `nodes`	list, get
`get_requests_limits`	`pods`	list
`list_pods`	`pods`	list
`get_pod_spec`	`pods`, `namespaces`	get
`check_security_context`	`pods`, `namespaces`	list, get
`find_latest_tags`	`pods`	list
`query_cpu_usage`, `query_memory_usage`	none (Prometheus HTTP API)	not applicable

```
rules:
  - apiGroups: [""]
    resources: ["nodes", "pods", "events", "namespaces"]
    verbs: ["get", "list", "watch"]
  - apiGroups: ["metrics.k8s.io"]
    resources: ["nodes", "pods"]
    verbs: ["get", "list"]
```

The role deliberately leaves out `nodes/proxy`, `pods/exec`, `pods/log` and `secrets`. Kubelet logs are not a tool; add `pods/log` only if you later want a pod-log tool.

Prerequisites in the cluster:

- metrics-server, only for the optional `usage` field in `describe_node`.
- Prometheus scraping cAdvisor (`container_cpu_usage_seconds_total`, `container_memory_working_set_bytes`) through the kubelet.
- kube-state-metrics, for `kube_pod_container_status_last_terminated_reason` and resource request metrics.
- A Prometheus URL reachable from the agent pod, set in a ConfigMap, plus a NetworkPolicy that allows that egress.

If `events.k8s.io/v1` is used instead of core events, add the `events.k8s.io` API group to the first rule and filter on `regarding.name` rather than `involvedObject.name`.